Exploratory Data Analysis — Phase 1

Introduction

After ingesting the cleaned datasets into a SQLite database, the next stage of the analytics workflow is Exploratory Data Analysis (EDA) combined with data engineering validation checks.

EDA helps analysts understand:

- data structure
- data quality
- distributions
- relationships between variables

However, in professional analytics pipelines, EDA is not only about visualization. It also includes data validation, schema inspection, and feature engineering.

Objectives of This Notebook

The goals of this notebook are to:

- connect to the SQLite database
- inspect database tables
- validate schemas
- detect missing values
- identify duplicates
- perform SQL-based exploratory analysis
- engineer new analytical features

Finally, we will create an analytical summary table called:

`customer_support_summary`

This table will contain cleaned and engineered variables optimized for analytics and BI dashboards.

In [1]:
```
# Import required libraries

import pandas as pd
import sqlite3
import numpy as np
```

Creating Database Connection

In order to perform SQL-based analysis, we first establish a connection to the SQLite database created in the previous notebook.

In [2]:
```
db_name = "customer_support.db"

conn = sqlite3.connect(db_name)

print("Database connection established.")
```

```
Database connection established.
```

Checking Tables in the Database

Before running analysis, we confirm that all required tables exist in the database.

Expected tables:

- customers
- orders
- agents
- interactions

In [3]:
```
query = "SELECT name FROM sqlite_master WHERE type='table';"

tables = pd.read_sql(query, conn)

tables
```

Out[3]:

	name
0	customers
1	orders
2	agents
3	interactions

Observations & Insights

Verify that the database contains the following tables:

- customers
- orders
- agents
- interactions

If any table is missing, the ingestion pipeline should be reviewed before proceeding.

Schema Inspection

Schema inspection allows us to understand:

- column names
- data types
- table structure

This step ensures that the tables were correctly created during ingestion.

In [4]:
```
pd.read_sql("PRAGMA table_info(customers);", conn)
```

Out[4]:

	cid	name	type	notnull	dflt_value	pk
0	0	unique_id	TEXT	0	None	0
1	1	customer_city	TEXT	0	None	0

In [5]:
```
pd.read_sql("PRAGMA table_info(orders);", conn)
```

Out[5]:

	cid	name	type	notnull	dflt_value	pk
0	0	order_id	TEXT	0	None	0
1	1	order_date_time	TEXT	0	None	0
2	2	item_price	REAL	0	None	0
3	3	product_category	TEXT	0	None	0

In [6]:
```
pd.read_sql("PRAGMA table_info(agents);", conn)
```

Out[6]:

	cid	name	type	notnull	dflt_value	pk
0	0	agent_name	TEXT	0	None	0
1	1	supervisor	TEXT	0	None	0
2	2	manager	TEXT	0	None	0
3	3	tenure_bucket	TEXT	0	None	0
4	4	agent_shift	TEXT	0	None	0

In [7]:
```
pd.read_sql("PRAGMA table_info(interactions);", conn)
```

Out[7]:

	cid	name	type	notnull	dflt_value	pk
0	0	unique_id	TEXT	0	None	0
1	1	channel_name	TEXT	0	None	0
2	2	category	TEXT	0	None	0
3	3	sub_category	TEXT	0	None	0
4	4	customer_remarks	TEXT	0	None	0
5	5	issue_reported_at	TEXT	0	None	0
6	6	issue_responded	TEXT	0	None	0
7	7	connected_handling_time	REAL	0	None	0
8	8	csat_score	INTEGER	0	None	0

Observations & Insights

From the schema inspection results, we can confirm that all four tables — **customers, orders, agents, and interactions** — were successfully created during the ingestion process and contain the expected columns. This indicates that the data normalization step was correctly implemented when splitting the original dataset into logical entities.

Customers Table The `customers` table contains two columns: `unique_id` and `customer_city`, both stored as `TEXT`.

- `unique_id` serves as the identifier for customers and will later allow us to link customer information with interaction records.
- The use of `TEXT` is appropriate because customer identifiers and city names are categorical values rather than numerical measures.

Orders Table The `orders` table includes four columns: `order_id`, `order_date_time`, `item_price`, and `product_category`.

- `order_id`, `order_date_time`, and `product_category` are stored as `TEXT`, which is suitable for identifiers, timestamps stored as strings, and categorical product classifications.
- `item_price` is stored as `REAL`, which correctly reflects that it is a numerical monetary value that can include decimals.

Agents Table The `agents` table contains `agent_name`, `supervisor`, `manager`, `tenure_bucket`, and `agent_shift`.

- All fields are stored as `TEXT`, which is appropriate because these represent categorical or descriptive attributes of agents.

- This table will be useful for analyzing performance across **agent hierarchy, experience levels, and shift patterns**.

Interactions Table The `interactions` table represents the core transactional dataset and includes nine columns.

- Identifiers and categorical fields such as `unique_id`, `channel_name`, `category`, `sub_category`, and `customer_remarks` are stored as `TEXT`, which is expected.
- Time-related fields `issue_reported_at` and `issue_responded` are also stored as `TEXT`, which is common when timestamps are initially ingested as strings and can later be converted to datetime format for time-based analysis.
- `connected_handling_time` is stored as `REAL`, reflecting a numerical measure of interaction duration.
- `csat_score` is stored as `INTEGER`, which is appropriate since customer satisfaction ratings are typically discrete numeric values.

Overall Assessment

- All expected columns are present across the tables.
- Data types are consistent with the nature of the variables.
- The schema confirms that the ingestion pipeline correctly created the normalized relational structure.

This successful schema validation indicates that the dataset is now properly structured for the next phase of analysis, including **data quality checks, feature engineering, and the construction of the analytical summary table.**

Basic Table Analysis Using SQL

We begin by examining the size of each table.

Counting rows helps us verify that ingestion preserved the dataset correctly.

In [8]:
```
pd.read_sql("SELECT COUNT(*) AS customers_count FROM customers", conn)
```

Out[8]:

	customers_count
0	85907

In [9]:
```
pd.read_sql("SELECT COUNT(*) AS orders_count FROM orders", conn)
```

Out[9]:

	orders_count
0	67676

In [10]:
```
pd.read_sql("SELECT COUNT(*) AS agents_count FROM agents", conn)
```

Out[10]:

	agents_count
0	1371

In [11]:
```
pd.read_sql("SELECT COUNT(*) AS interactions_count FROM interactions", conn)
```

Out[11]:

	interactions_count
0	85907

Observations & Insights

Compare the counts of the tables and note whether the number of rows aligns with expectations.

DISTINCT Value Analysis

Understanding unique values helps identify:

- categorical variables
- potential data inconsistencies
- category distributions

In [12]:
```
pd.read_sql("SELECT DISTINCT category FROM interactions", conn)
```

Out[12]:

	category
0	Product Queries
1	Order Related
2	Returns
3	Cancellation
4	Shopzilla Related
5	Payments related
6	Refund Related
7	Feedback
8	Offers & Cashback
9	Onboarding related
10	Others
11	App/website

In [13]:
```
pd.read_sql("SELECT DISTINCT product_category FROM orders", conn)
```

Out[13]:

	product_category
0	None
1	LifeStyle
2	Electronics
3	Mobile
4	Home Appliences
5	Furniture
6	Home
7	Books & General merchandise
8	GiftCard
9	Affiliates

In [14]:
```
pd.read_sql("SELECT DISTINCT product_category FROM orders", conn)
```

Out[14]:

	product_category
0	None
1	LifeStyle
2	Electronics
3	Mobile
4	Home Appliences
5	Furniture
6	Home
7	Books & General merchandise
8	GiftCard
9	Affiliates

In [15]:
```
pd.read_sql("SELECT DISTINCT agent_shift FROM agents", conn)
```

Out[15]:

	agent_shift
0	Morning
1	Evening
2	Split
3	Afternoon
4	Night

Observations & Insights

The DISTINCT value analysis helps us understand the categorical structure of the dataset and identify any inconsistencies or anomalies in categorical fields. Reviewing the results from the `interactions`, `orders`, and `agents` tables reveals several important observations.

Interaction Categories

The `category` column in the **interactions** table contains 12 distinct categories such as *Product Queries, Order Related, Returns, Cancellation, Payments related,* and *Refund Related*. These categories represent the primary reasons customers contact support. Overall, the categories appear logically grouped around key operational areas such as **orders, payments, returns, and technical issues**.

However, some minor formatting inconsistencies are noticeable:

- Some categories use **title case** (e.g., *Product Queries*), while others use **lowercase words** (e.g., *Payments related*).
- Variations like **"Refund Related"** and **"Returns"** may represent similar operational areas but are categorized separately.
- The category **"Others"** is very broad and may require further investigation to determine whether it hides meaningful sub-issues.

These inconsistencies may not break analysis but should be kept in mind when grouping categories during later analysis.

Product Categories

The `product_category` column in the **orders** table contains several product groups including *Electronics, Mobile, Furniture, LifeStyle,* and *Home Appliances*. These categories represent the types of products associated with customer orders.

Key observations include:

- A **"None"** category exists, indicating that some records do not have a valid product category assigned. This could represent missing or incomplete data.
- The category **"Home Appliences"** contains a spelling error and should likely be standardized to **"Home Appliances"**.
- Categories such as **"Books & General merchandise"** and **"GiftCard"** may require consistent formatting (e.g., spacing or capitalization) if used for grouped analysis later.

These issues should be corrected during the data cleaning phase to ensure consistent category grouping.

Agent Shifts

The `agent_shift` column in the **agents** table contains five distinct shifts:

- Morning
- Afternoon
- Evening
- Night
- Split

These shift categories appear consistent and clearly represent operational work schedules. The presence of a **Split shift** suggests some agents may work across multiple time periods, which could be interesting to analyze when evaluating **agent productivity or customer satisfaction across shifts**.

Overall Assessment

The categorical variables appear well-defined and meaningful for business analysis. However, minor issues such as **spelling errors, inconsistent capitalization, and missing**

categories ("None") were identified. Addressing these inconsistencies during data cleaning will improve the reliability of downstream analyses such as **grouped aggregations, performance comparisons, and dashboard visualizations**.

GROUP BY Analysis

Group-by analysis helps summarize patterns within the dataset.

Examples include:

- number of tickets per category
- workload distribution by agent shift

In [16]:
```
pd.read_sql("""
SELECT category, COUNT(*) AS ticket_count
FROM interactions
GROUP BY category
ORDER BY ticket_count DESC
""", conn)
```

Out[16]:

	category	ticket_count
0	Returns	44097
1	Order Related	23215
2	Refund Related	4550
3	Product Queries	3692
4	Shopzilla Related	2792
5	Payments related	2327
6	Feedback	2294
7	Cancellation	2212
8	Offers & Cashback	480
9	Others	99
10	App/website	84
11	Onboarding related	65

In [17]:
```
pd.read_sql("""
SELECT agent_shift, COUNT(*) AS shift_ticket_count
FROM agents
GROUP BY agent_shift
""", conn)
```

Out[17]:

	agent_shift	shift_ticket_count
0	Afternoon	96
1	Evening	526
2	Morning	670
3	Night	18
4	Split	61

Observations & Insights

The **GROUP BY analysis** helps identify operational patterns within the dataset by summarizing support interactions by **issue category** and examining the **distribution of agents across shifts**.

Ticket Distribution by Issue Category

The results show that customer support interactions are heavily concentrated in a few categories.

- **Returns** is by far the largest category with **44,097 tickets**, accounting for a substantial portion of all support interactions. This indicates that product returns represent the most significant operational challenge and may require improvements in return policies, product descriptions, or quality control.
- **Order Related issues** are the second largest category with **23,215 tickets**, suggesting that many customers contact support regarding order tracking, delivery issues, or order modifications.
- Mid-volume categories include **Refund Related (4,550)** and **Product Queries (3,692)**, which indicate that customers frequently seek clarification about refunds or product details.
- Lower-volume categories such as **Offers & Cashback**, **Others**, **App/website**, and **Onboarding related** generate significantly fewer interactions. These may represent niche issues or specialized support areas.

Overall, the distribution shows that **Returns and Order-related concerns dominate customer support demand**, which highlights key areas where operational improvements could significantly reduce ticket volume.

Agent Shift Distribution

The shift distribution shows how support agents are allocated across different working hours.

- The **Morning shift** has the largest number of agents (**670**), followed by the **Evening shift (526)** and **Afternoon shift (96)**. This suggests that staffing is concentrated during daytime hours, likely aligning with peak customer activity periods.
- The **Split shift (61)** indicates some agents work across multiple time blocks, potentially to cover fluctuating demand.
- The **Night shift** has the fewest agents (**18**), which is expected since customer support demand typically decreases during late-night hours.

Operational Implications

- Since **Returns and Order-related queries dominate ticket volume**, these areas represent the biggest opportunities for improving customer experience through better order tracking, clearer return policies, and proactive communication.
- Staffing patterns appear aligned with typical business hours, but further analysis could compare **ticket volume by time of day** to determine whether staffing levels optimally match customer demand.

These findings will guide deeper analysis in later stages, particularly when examining **customer satisfaction (CSAT), resolution times, and agent performance across categories and shifts**.

Missing Value Analysis

Missing values can significantly affect analytics results.

We inspect each table for missing values to determine whether imputation or filtering may be required.

In [18]:
```
interactions = pd.read_sql("SELECT * FROM interactions", conn)

interactions.isnull().sum()
```

Out[18]:
```
unique_id                     0
channel_name                  0
category                      0
sub_category                  0
customer_remarks          57165
issue_reported_at         53933
issue_responded           54022
connected_handling_time   85665
csat_score                    0
dtype: int64
```

Observations & Insights

The missing value analysis reveals that several columns in the **interactions** table contain a significant number of null values, while others are fully populated.

Columns Without Missing Values

The following columns have **no missing values**, indicating that they were consistently recorded during data collection:

- `unique_id`
- `channel_name`
- `category`
- `sub_category`
- `csat_score`

These fields are critical identifiers and categorical variables used in most analytical queries. Their completeness ensures that **core interaction categorization and satisfaction analysis can be performed reliably**.

Columns With Missing Values

Several columns contain substantial missing values:

- **customer_remarks — 57,165 missing values**
  This suggests that in many interactions customers did not leave textual feedback. This is common in support datasets because remarks are typically optional fields. While this limits text-based sentiment analysis for some records, it does not prevent other types of analysis such as CSAT or category-based insights.

- **issue_reported_at — 53,933 missing values**
  Missing timestamps may prevent accurate calculation of response times and resolution durations for those records.

- **issue_responded — 54,022 missing values**
  Similar to the reporting timestamp, missing response timestamps can limit the ability to measure **resolution time** or **agent responsiveness**.

- **connected_handling_time — 85,665 missing values**
  This column contains the largest number of missing values, suggesting that handling time may only be recorded for specific interaction types (e.g., calls) and not for others (e.g., emails or asynchronous communication channels).

Analytical Implications

These missing values will influence certain analyses:

- **Time-based metrics** such as response time or resolution duration may only be calculated for records where both timestamps are available.
- **Handling time analysis** may require filtering out records with missing values.
- **Text analysis or sentiment modeling** may only be possible on interactions where customer remarks are present.

Next Steps

To ensure reliable analysis in later stages, we will:

- Convert timestamp fields to proper datetime format.
- Create derived metrics such as **Resolution Time** only for records with valid timestamps.
- Apply filtering or conditional calculations when analyzing **handling time**.
- Retain records with missing remarks but treat them separately when performing **text-based analysis**.

Addressing these missing values carefully will ensure that the subsequent analytical summary table remains **accurate, consistent, and suitable for downstream analysis and visualization**.

Duplicate Detection

Duplicate records can lead to incorrect analytics results.

We verify whether duplicate interaction records exist.

In [20]:
```
interactions.duplicated().sum()
```

Out[20]:
```
np.int64(0)
```

Observations & Insights

The duplicate detection check shows that the **interactions table contains 0 duplicate records**, indicating that each row represents a unique interaction entry in the dataset.

This result suggests that the data ingestion and table creation processes preserved the integrity of the original dataset without introducing duplicate rows. The absence of duplicates is important because duplicate interaction records could artificially inflate metrics such as ticket counts, issue frequencies, or agent workload.

Since no duplicates were detected:

- No deduplication or record filtering is required at this stage.

- Aggregation queries (such as ticket counts by category or agent performance metrics) can be performed confidently without risk of double counting.
- The dataset maintains a reliable one-record-per-interaction structure.

Overall, this confirms that the dataset is structurally clean with respect to record duplication, allowing us to proceed with feature engineering and further exploratory analysis without additional duplicate handling steps.

Data Cleaning Based on EDA Findings

During the exploratory data analysis phase, several data quality issues were identified through:

- DISTINCT value analysis
- missing value analysis

The key issues discovered included:

- inconsistent category formatting
- spelling errors in categorical variables
- placeholder values such as "None"
- missing timestamp values
- missing handling time records

Cleaning the dataset before feature engineering is a critical step in analytics pipelines because it ensures that derived metrics and downstream analysis remain accurate, consistent, and reliable.

In this section, we apply targeted data cleaning operations based on the insights identified earlier.

Standardizing Category Values

The DISTINCT value analysis revealed that some categorical fields contain formatting inconsistencies such as:

- inconsistent capitalization
- extra whitespace characters
- inconsistent naming patterns

These inconsistencies can lead to incorrect grouping during aggregation queries and visualizations.

To address this, we standardize categorical values by:

- removing extra whitespace
- standardizing capitalization
- correcting known category inconsistencies

In [50]:
```
# 1 — Load Tables from SQLite

import pandas as pd
import numpy as np

# Load tables from SQLite database
interactions = pd.read_sql("SELECT * FROM interactions", conn)
orders = pd.read_sql("SELECT * FROM orders", conn)
agents = pd.read_sql("SELECT * FROM agents", conn)
```

Cleaning Interaction Categories

The interactions table contains categorical variables such as:

- `category`
- `sub_category`

These fields are important for analyzing customer support issue types.

We standardize them by:

- removing extra whitespace
- normalizing capitalization

In [51]:
```
# 2 — Standardize Interaction Categories

# Remove extra spaces
interactions["category"] = interactions["category"].str.strip()
interactions["sub_category"] = interactions["sub_category"].str.strip()

# Standardize capitalization
interactions["category"] = interactions["category"].str.title()
interactions["sub_category"] = interactions["sub_category"].str.title()
```

Correcting Known Category Formatting Issues

During DISTINCT value analysis, certain category values were identified as having minor inconsistencies in formatting.

To maintain consistency across the dataset, we standardize these category labels.

In [52]:
```
# 3 — Fix Known Category Formatting Issues

interactions["category"] = interactions["category"].replace({
    "Payments Related": "Payments Related",
```

```
    "Refund Related": "Refund Related",
    "App/Website": "App/Website"
})
```

Cleaning Product Categories

The DISTINCT analysis revealed several issues in the product_category column of the orders table:

- spelling error: "Home Appliences"
- placeholder category "None"
- inconsistent formatting for some product types

These inconsistencies can affect product-level analysis and grouping.

We address these issues by:

- removing extra whitespace
- correcting spelling errors
- converting placeholder values to missing values
- standardizing capitalization

In [53]:
```
# 4 — Clean Product Categories

# Remove extra spaces
orders["product_category"] = orders["product_category"].str.strip()

# Fix spelling errors and placeholder values
orders["product_category"] = orders["product_category"].replace({
    "Home Appliences": "Home Appliances",
    "None": np.nan
})

# Standardize capitalization
orders["product_category"] = orders["product_category"].str.title()
```

Standardizing Agent Shift Values

The agent_shift column represents the working schedule of customer support agents.

To ensure consistent grouping during analysis, we standardize this field by:

- removing whitespace
- normalizing capitalization

In [54]:
```
# 5 — Standardize Agent Shift Values

# Remove whitespace
agents["agent_shift"] = agents["agent_shift"].str.strip()
```

```
# Standardize casing
agents["agent_shift"] = agents["agent_shift"].str.title()
```

Writing Cleaned Tables Back to SQLite

After applying the cleaning transformations, the updated tables must be written back into the SQLite database.

This step is essential because all SQL queries in subsequent stages will run directly on the database.

In [55]:
```
# 6 — Write Cleaned Tables Back to SQLite

interactions.to_sql("interactions", conn, if_exists="replace", index=False)
orders.to_sql("orders", conn, if_exists="replace", index=False)
agents.to_sql("agents", conn, if_exists="replace", index=False)
```

Out[55]: 1371

Verifying Data Cleaning Results

To confirm that the cleaning steps were successful, we run DISTINCT queries again to inspect the categorical values stored in the database.

This ensures that:

- spelling errors have been corrected
- categories are consistently formatted
- no unexpected values remain

In [56]:
```
# 7 — Verify Interaction Categories

pd.read_sql("""
SELECT DISTINCT category
FROM interactions
ORDER BY category
""", conn)
```

Out[56]:

	category
0	App/Website
1	Cancellation
2	Feedback
3	Offers & Cashback
4	Onboarding Related
5	Order Related
6	Others
7	Payments Related
8	Product Queries
9	Refund Related
10	Returns
11	Shopzilla Related

In [57]:

```
# Verify Product Categories

pd.read_sql("""
SELECT DISTINCT product_category
FROM orders
ORDER BY product_category
""", conn)
```

Out[57]:

	product_category
0	None
1	Affiliates
2	Books & General Merchandise
3	Electronics
4	Furniture
5	Gift Card
6	Home
7	Home Appliances
8	Lifestyle
9	Mobile

In [58]:

```
# Verify Agent Shifts
```

```
pd.read_sql("""
SELECT DISTINCT agent_shift
FROM agents
ORDER BY agent_shift
""", conn)
```

Out[58]:

	agent_shift
0	Afternoon
1	Evening
2	Morning
3	Night
4	Split

Observations & Insights

The verification queries confirm that the data cleaning operations successfully standardized categorical variables across the dataset.

Key improvements include:

- correction of spelling errors such as "Home Appliences" → "Home Appliances"
- conversion of placeholder category values ("None") to proper missing values
- removal of extra whitespace characters
- consistent capitalization across categorical fields

After these cleaning steps, the categorical variables are now consistent and reliable for aggregation, grouping, and visualization.

These improvements will ensure that downstream analytics, feature engineering, and dashboard visualizations produce accurate and meaningful results.

Handling Missing Values and Safe Feature Engineering

During earlier exploratory analysis, we identified several columns in the interactions table containing missing values, including:

- customer_remarks
- issue_reported_at
- issue_responded
- connected_handling_time

In analytics pipelines, missing values are not always errors. They often carry meaningful information about system behavior or data collection processes.

For example:

- Missing handling time may indicate non-call interactions
- Missing remarks may indicate customers did not provide written feedback
- Missing timestamps may indicate incomplete system logs

Therefore, instead of blindly filling missing values, we apply careful transformations that preserve analytical integrity.

Strategy

The following steps will be applied:

- Convert timestamp columns to datetime format
- Create time-based features only when timestamps exist
- Handle handling-time inconsistencies safely
- Create categorical CSAT segments
- Track presence of customer remarks
- Preserve missing values when they represent real operational conditions

This approach ensures that downstream metrics such as resolution time and handling time remain accurate and meaningful.

In [59]:
```
# 1 — Load Interactions Table

interactions = pd.read_sql("SELECT * FROM interactions", conn)

interactions.head()
```

Out[59]:

	unique_id	channel_name	category	sub_category	customer_remarks	issue_rep
0	7e9ae164-6a8b-4521-a2d4-58f7c9fff13f	Outcall	Product Queries	Life Insurance	None	20
1	b07ec1b0-f376-43b6-86df-ec03da3b2e16	Outcall	Product Queries	Product Specific Information	None	20
2	200814dd-27c7-4149-ba2b-bd3af3092880	Inbound	Order Related	Installation/Demo	None	20
3	eb0d3e53-c1ca-42d3-8486-e42c8d622135	Inbound	Returns	Reverse Pickup Enquiry	None	20
4	ba903143-1e54-406c-b969-46c52f92e5df	Inbound	Cancellation	Not Needed	None	20

Inspect Missing Values

Before applying transformations, we inspect the distribution of missing values in the dataset.

Understanding the pattern of missing values helps determine which variables require special handling during feature engineering.

In [60]:

```
# 2 — Inspect Missing Values

interactions.isnull().sum()
```

Out[60]:

```
unique_id                     0
channel_name                  0
category                      0
sub_category                  0
customer_remarks          57165
issue_reported_at         53933
issue_responded           54022
connected_handling_time   85665
csat_score                    0
dtype: int64
```

Observations & Insights

The missing value summary confirms the earlier EDA findings:

- A large number of records lack customer remarks
- Several records contain missing timestamp values
- Handling time is missing for many interactions

These missing values will be handled carefully to ensure that derived metrics remain analytically valid.

Converting Timestamp Columns

Many timestamp fields stored in SQLite are read as text strings when loaded into pandas.

To perform time-based calculations, these columns must be converted into datetime format.

We use errors="coerce" so that invalid timestamps are converted to NaT (Not a Time) instead of causing runtime errors.

In [61]:
```
# 3 — Convert Timestamp Columns to Datetime

interactions["issue_reported_at"] = pd.to_datetime(
    interactions["issue_reported_at"], errors="coerce"
)

interactions["issue_responded"] = pd.to_datetime(
    interactions["issue_responded"], errors="coerce"
)
```

Creating Resolution Time Feature

Resolution time represents the duration between issue reporting and issue response.

This metric is a key indicator of customer support efficiency.

Resolution time is calculated only when both timestamps are available.

Rows with missing timestamps will remain NaN, which is intentional and preserves analytical accuracy.

In [62]:
```
# 4 — Create Resolution Time Feature

interactions["resolution_time_minutes"] = (
    interactions["issue_responded"] - interactions["issue_reported_at"]
).dt.total_seconds() / 60
```

Cleaning Handling Time Values

The connected_handling_time column records the duration an agent spent handling an interaction.

However, data inconsistencies such as negative values or invalid entries may appear in operational datasets.

We clean this column by replacing invalid values with NaN, ensuring that analytical calculations remain reliable.

In [63]:
```
# 5 — Clean Handling Time Column

interactions["connected_handling_time"] = interactions["connected_handling_time"].a
    lambda x: x if pd.notnull(x) and x >= 0 else np.nan
)
```

Creating Handling Time Minutes Feature

The dataset already stores handling time in minutes, so this feature simply standardizes the column name used for analysis.

This metric will later be used for:

- agent productivity analysis
- operational efficiency metrics
- workload comparisons

In [64]:
```
# 6 — Create Handling Time Minutes Column

interactions["handling_time_minutes"] = interactions["connected_handling_time"]
```

Creating Customer Satisfaction Categories

The dataset contains a numerical CSAT score representing customer satisfaction levels.

To improve interpretability in analytics and dashboards, we convert numeric scores into categorical satisfaction levels.

In [65]:
```
# 7 — Create CSAT Category

def csat_category(score):
    if score >= 4:
        return "Satisfied"
    elif score == 3:
        return "Neutral"
    else:
```

```
        return "Dissatisfied"

interactions["csat_category"] = interactions["csat_score"].apply(csat_category)
```

Handling Missing Customer Remarks

Customer remarks are optional textual feedback fields.

Instead of removing rows with missing remarks, we create an indicator variable that identifies whether a remark exists.

This enables analysis such as:

- satisfaction differences between feedback vs no-feedback interactions
- remark participation rates

In [66]:
```
# 8 — Create Remark Indicator

interactions["has_customer_remark"] = interactions["customer_remarks"].notnull().as
```

Creating Interaction Type Feature

The channel_name column identifies the communication channel used by the customer.

We standardize this into a new analytical feature called interaction_type, which can be used to analyze:

- channel performance
- customer engagement patterns
- response behavior across communication channels

In [67]:
```
# 9 — Create Interaction Type Feature

interactions["interaction_type"] = interactions["channel_name"].str.title()
```

Preview Engineered Dataset

After applying the feature engineering steps, we inspect the dataset to confirm that the new columns were created correctly.

In [68]:
```
# 10 — Check Result

interactions.head()
```

Out[68]:

	unique_id	channel_name	category	sub_category	customer_remarks	issue_rep
0	7e9ae164-6a8b-4521-a2d4-58f7c9fff13f	Outcall	Product Queries	Life Insurance	None	20
1	b07ec1b0-f376-43b6-86df-ec03da3b2e16	Outcall	Product Queries	Product Specific Information	None	20
2	200814dd-27c7-4149-ba2b-bd3af3092880	Inbound	Order Related	Installation/Demo	None	20
3	eb0d3e53-c1ca-42d3-8486-e42c8d622135	Inbound	Returns	Reverse Pickup Enquiry	None	20
4	ba903143-1e54-406c-b969-46c52f92e5df	Inbound	Cancellation	Not Needed	None	20

Verifying Missing Values After Feature Engineering

We check the dataset again to ensure that missing values were handled appropriately.

Certain columns will still contain missing values, which is expected.

In [69]:

```
# 11 — Verify Missing Values After Engineering

interactions.isnull().sum()
```

Out[69]:

```
unique_id                     0
channel_name                  0
category                      0
sub_category                  0
customer_remarks          57165
issue_reported_at         53933
issue_responded           54022
connected_handling_time   85665
csat_score                    0
resolution_time_minutes   54274
handling_time_minutes     85665
csat_category                 0
has_customer_remark           0
interaction_type              0
dtype: int64
```

Observations & Insights

After feature engineering, missing values remain in the following columns:

- customer_remarks
- resolution_time_minutes
- handling_time_minutes

These missing values are intentional and analytically meaningful:

- Missing remarks indicate customers who did not provide textual feedback
- Missing resolution time occurs when timestamps are unavailable
- Missing handling time indicates interactions where handling duration was not recorded

Preserving these missing values ensures that downstream analytics reflect real operational conditions rather than artificial imputation.

Handling Placeholder Values in Customer Remarks

During additional inspection of the dataset, it was observed that some records in the customer_remarks column contain the string value "None" instead of a true missing value (NULL / NaN).

This type of placeholder value is common in operational datasets where systems store missing text fields as literal strings.

If left uncorrected, these placeholder values can lead to incorrect interpretations, such as treating "None" as actual customer feedback.

Therefore, we convert these placeholder values into true missing values to maintain analytical accuracy.

In [70]:
```
# 12 — Reload Interactions Table

interactions = pd.read_sql("SELECT * FROM interactions", conn)

interactions.head()
```

Out[70]:

	unique_id	channel_name	category	sub_category	customer_remarks	issue_rep
0	7e9ae164-6a8b-4521-a2d4-58f7c9fff13f	Outcall	Product Queries	Life Insurance	None	20
1	b07ec1b0-f376-43b6-86df-ec03da3b2e16	Outcall	Product Queries	Product Specific Information	None	20
2	200814dd-27c7-4149-ba2b-bd3af3092880	Inbound	Order Related	Installation/Demo	None	20
3	eb0d3e53-c1ca-42d3-8486-e42c8d622135	Inbound	Returns	Reverse Pickup Enquiry	None	20
4	ba903143-1e54-406c-b969-46c52f92e5df	Inbound	Cancellation	Not Needed	None	20

Inspecting Customer Remarks Values

Before applying corrections, we inspect the distribution of values in the customer_remarks column.

This helps confirm whether placeholder values such as "None" exist within the dataset.

In [71]:

```
# 13 — Inspect Customer Remarks Values

interactions["customer_remarks"].value_counts(dropna=False).head(10)
```

Out[71]:

```
customer_remarks
None          57165
Good           1390
Good           1158
Very good       569
Nice            316
Thanks          276
Ok              259
No              258
Thank you       244
Nice            239
Name: count, dtype: int64
```

Observations & Insights

The value distribution helps identify whether "None" appears as a literal string instead of representing a true missing value.

If such placeholders exist, they must be converted to proper missing values (NaN) so that analytical calculations correctly interpret the absence of remarks.

Converting Placeholder Values to Missing Values

To ensure consistent handling of missing remarks, we replace the placeholder string "None" with NaN.

This allows pandas and SQL queries to correctly recognize these entries as missing data.

In [72]:
```
# 14 — Convert Placeholder Strings to NULL

import numpy as np

interactions["customer_remarks"] = interactions["customer_remarks"].replace("None",
```

Recomputing the Remark Indicator Column

Earlier in the feature engineering process, we created a column called: `has_customer_remark`

This variable indicates whether a customer provided written feedback.

Since the placeholder "None" values have now been converted to NaN, we must recalculate the indicator column to ensure it reflects the correct remark status.

In [73]:
```
# 15 — Recalculate Remark Indicator Column

interactions["has_customer_remark"] = interactions["customer_remarks"].notnull().as
```

Verifying Missing Values After Correction

After converting placeholder values and recomputing the indicator column, we inspect the dataset again to confirm that the missing value structure is correct.

In [74]:
```
# 16 — Validate Missing Values Again

interactions.isnull().sum()
```

Out[74]:
```
unique_id                     0
channel_name                  0
category                      0
sub_category                  0
customer_remarks          57165
issue_reported_at         53933
issue_responded           54022
connected_handling_time   85665
csat_score                    0
has_customer_remark           0
dtype: int64
```

Observations & Insights

The updated missing value summary should confirm that:

- customer_remarks remains missing where no remarks were provided
- has_customer_remark contains no missing values
- the indicator column correctly reflects the presence or absence of feedback

---

Validating the Remark Indicator Logic

To ensure the indicator column was computed correctly, we review several rows of the dataset.

This allows us to confirm the relationship between:

- `customer_remarks`
- `has_customer_remark`

In [75]:
```
# 17 — Verify Remark Indicator

interactions[["customer_remarks", "has_customer_remark"]].head(10)
```

Out[75]:

	customer_remarks	has_customer_remark
0	None	0
1	None	0
2	None	0
3	None	0
4	None	0
5	None	0
6	None	0
7	Very good	1
8	Shopzilla app and it's all coustomer care serv...	1
9	None	0

Observations & Insights

The sample rows confirm that:

- records with NULL remarks have has_customer_remark = 0
- records with text remarks have has_customer_remark = 1

This indicator column will be useful for analyzing feedback participation patterns and customer engagement levels.

Writing the Cleaned Table to SQLite

After completing all corrections and feature engineering steps, the cleaned dataset is written back into the SQLite database.

This ensures that downstream analyses and SQL queries operate on a fully cleaned and engineered dataset.

The new table is stored as: `interactions_cleaned`

In [76]:

```
# 18 — Write Cleaned Table Back to SQLite

interactions.to_sql(
    "interactions_cleaned",
    conn,
    if_exists="replace",
```

```
    index=False
)
```

Out[76]: 85907

Confirming Table Creation

Finally, we verify that the cleaned table has been successfully stored in the database.

In [77]:
```
# 19 — Confirm Table Creation

pd.read_sql("SELECT COUNT(*) FROM interactions_cleaned", conn)
```

Out[77]:

	COUNT(*)
0	85907

In [79]:
```
interactions_cleaned = pd.read_sql(
    "SELECT * FROM interactions_cleaned",
    conn
)

interactions_cleaned.head()
```

Out[79]:

	unique_id	channel_name	category	sub_category	customer_remarks	issue_rep
0	7e9ae164-6a8b-4521-a2d4-58f7c9fff13f	Outcall	Product Queries	Life Insurance	None	20
1	b07ec1b0-f376-43b6-86df-ec03da3b2e16	Outcall	Product Queries	Product Specific Information	None	20
2	200814dd-27c7-4149-ba2b-bd3af3092880	Inbound	Order Related	Installation/Demo	None	20
3	eb0d3e53-c1ca-42d3-8486-e42c8d622135	Inbound	Returns	Reverse Pickup Enquiry	None	20
4	ba903143-1e54-406c-b969-46c52f92e5df	Inbound	Cancellation	Not Needed	None	20

Observations & Insights

The successful query result confirms that the interactions_cleaned table has been written to the database.

This table now represents the fully cleaned interaction dataset, containing:

- corrected categorical values
- validated timestamp fields
- engineered analytical features
- accurate missing-value handling

This dataset will serve as the primary interaction data source for creating the analytical summary table in the next stage of the pipeline.

Final Standardization of Customer Remarks

Background

During validation of the customer_remarks column, two remaining inconsistencies were identified:

- The value "None" still appeared as a literal string instead of a true missing value.
- Certain remark values appeared duplicated due to formatting differences such as:
  - "Good"
  - " Good"
  - "Good "
  - "GOOD"

These inconsistencies occur frequently in operational datasets where user-generated text may contain extra spaces, inconsistent capitalization, or placeholder strings.

If left unresolved, these issues can lead to incorrect counts when performing analyses such as:

- feedback frequency analysis
- sentiment grouping
- remark-based customer engagement metrics

To resolve this, we perform a final standardization of the customer_remarks column.

Cleaning Strategy

The cleaning process follows four steps:

- Convert remarks to string format to ensure consistent processing.
- Remove leading and trailing whitespace.
- Standardize capitalization.
- Convert placeholder values such as "None" into proper missing values (NaN).

This ensures that remark values are consistent and analytically reliable.

In [83]:
```
# 1 — Load Cleaned Interactions Table

interactions_cleaned = pd.read_sql(
    "SELECT * FROM interactions_cleaned",
    conn
)

interactions_cleaned.head()
```

Out[83]:

	unique_id	channel_name	category	sub_category	customer_remarks	issue_rep
0	7e9ae164-6a8b-4521-a2d4-58f7c9fff13f	Outcall	Product Queries	Life Insurance	None	20
1	b07ec1b0-f376-43b6-86df-ec03da3b2e16	Outcall	Product Queries	Product Specific Information	None	20
2	200814dd-27c7-4149-ba2b-bd3af3092880	Inbound	Order Related	Installation/Demo	None	20
3	eb0d3e53-c1ca-42d3-8486-e42c8d622135	Inbound	Returns	Reverse Pickup Enquiry	None	20
4	ba903143-1e54-406c-b969-46c52f92e5df	Inbound	Cancellation	Not Needed	None	20

Inspect Customer Remark Values

Before applying corrections, we inspect the value distribution of the customer_remarks column to confirm the presence of placeholder values and formatting inconsistencies.

In [84]:
```
# 2 — Inspect Customer Remarks Distribution

interactions_cleaned["customer_remarks"].value_counts(dropna=False).head(10)
```

Out[84]:
```
customer_remarks
None          57165
Good           1390
Good           1158
Very good       569
Nice            316
Thanks          276
Ok              259
No              258
Thank you       244
Nice            239
Name: count, dtype: int64
```

Standardizing Remark Formatting

User-entered text fields often contain inconsistencies such as:

- extra whitespace
- inconsistent capitalization

To ensure consistent grouping during analysis, we normalize remark formatting by:

- converting values to string format
- removing leading and trailing whitespace
- standardizing capitalization

In [85]:
```
# 3 — Standardize Remark Formatting

interactions_cleaned["customer_remarks"] = interactions_cleaned["customer_remarks"]

interactions_cleaned["customer_remarks"] = interactions_cleaned["customer_remarks"]

interactions_cleaned["customer_remarks"] = interactions_cleaned["customer_remarks"]
```

Converting Placeholder Values to Missing Values

Operational systems sometimes store missing remarks as string placeholders such as:

- None
- Nan
- ""

These values should be converted to proper missing values (NaN) to ensure accurate analytics.

In [86]:
```
# 4 — Convert Placeholder Strings to NULL

interactions_cleaned["customer_remarks"] = interactions_cleaned["customer_remarks"]
    "None": np.nan,
    "Nan": np.nan,
    "": np.nan
})
```

Recomputing the Remark Indicator

Because the remark column has been cleaned, the indicator variable that tracks whether a customer provided feedback must be recalculated.

This column will be useful for analyzing:

- customer feedback participation rates
- relationship between remarks and CSAT scores

In [87]:
```
# 5 — Recalculate Indicator Column

interactions_cleaned["has_customer_remark"] = (
    interactions_cleaned["customer_remarks"]
    .notnull()
    .astype(int)
)
```

Verifying Cleaning Results

To confirm that the cleaning process was successful, we review the most frequent remark values again.

In [88]:
```
# Cell 6 — Verify Cleaning

interactions_cleaned["customer_remarks"].value_counts(dropna=False).head(10)
```

Out[88]:
```
customer_remarks
NaN             57171
Good             2680
Very Good         860
Nice              578
Thanks            455
Thank You         366
No                297
Ok                282
Excellent         236
Good Service      210
Name: count, dtype: int64
```

Observations & Insights

The verification results confirm that the remark column has been successfully standardized.

Key improvements include:

- placeholder values such as "None" have been converted to proper missing values
- duplicate remark entries caused by formatting inconsistencies have been merged
- capitalization is now consistent across all remark values

These improvements ensure that remark-based analyses produce accurate and meaningful insights.

Saving the Updated Clean Table

After applying the final cleaning steps, we write the updated dataset back to SQLite.

This ensures that all downstream SQL queries operate on the latest cleaned interaction dataset.

In [89]:
```
# 7 — Save Updated Clean Table

interactions_cleaned.to_sql(
    "interactions_cleaned",
    conn,
    if_exists="replace",
    index=False
)
```

Out[89]: 85907

Confirm Table Update

Finally, we confirm that the updated table was successfully written to the database.

In [90]:
```
# 8 — Confirm Table Exists

pd.read_sql("SELECT COUNT(*) FROM interactions_cleaned", conn)
```

Out[90]:

	COUNT(*)
0	85907

Updating the Database Schema

During the data cleaning stage, a new table called: `interactions_cleaned`

was created containing the fully cleaned and standardized interaction dataset.

To simplify the database structure and remove redundancy, we will:

- Delete the original `interactions` table
- Rename `interactions_cleaned` to `interactions`

This ensures that the database contains only the final cleaned interaction dataset, which will be used for all subsequent analytics queries.

This approach is common in analytics pipelines where cleaned tables replace raw operational tables after validation.

In [91]:
```
# Drop the old interactions table

conn.execute("DROP TABLE IF EXISTS interactions")
conn.commit()
```

Renaming the Cleaned Table

After removing the original table, we rename the cleaned table so that it becomes the primary interaction dataset in the database.

In [92]:
```
# Rename cleaned table to interactions

conn.execute("""
ALTER TABLE interactions_cleaned
RENAME TO interactions
""")

conn.commit()
```

In [93]:
```
pd.read_sql(
    "SELECT name FROM sqlite_master WHERE type='table'",
    conn
)
```

Out[93]:

	name
0	customers
1	orders
2	agents
3	interactions

Inspecting Tables After Data Cleaning

After completing the data cleaning and transformation steps, it is important to inspect the contents of each table to ensure that no inconsistencies remain.

This step helps verify:

- column names and structure
- cleaned categorical values
- correct formatting of timestamps
- removal of placeholder values such as "None"
- successful feature engineering

We will preview the first few rows of each table in the SQLite database.

In [94]:
```
pd.read_sql("""
SELECT *
FROM customers
LIMIT 5
""", conn)
```

Out[94]:

	unique_id	customer_city
0	7e9ae164-6a8b-4521-a2d4-58f7c9fff13f	None
1	b07ec1b0-f376-43b6-86df-ec03da3b2e16	None
2	200814dd-27c7-4149-ba2b-bd3af3092880	None
3	eb0d3e53-c1ca-42d3-8486-e42c8d622135	None
4	ba903143-1e54-406c-b969-46c52f92e5df	None

In [95]:
```
pd.read_sql("""
SELECT *
FROM orders
LIMIT 5
""", conn)
```

Out[95]:

	order_id	order_date_time	item_price	product_category
0	c27c9bb4-fa36-4140-9f1f-21009254ffdb	None	None	None
1	d406b0c7-ce17-4654-b9de-f08d421254bd	None	None	None
2	c273368d-b961-44cb-beaf-62d6fd6c00d5	None	None	None
3	5aed0059-55a4-4ec6-bb54-97942092020a	None	None	None
4	e8bed5a9-6933-4aff-9dc6-ccefd7dcde59	None	None	None

In [96]:

```
pd.read_sql("""
SELECT *
FROM agents
LIMIT 5
""", conn)
```

Out[96]:

	agent_name	supervisor	manager	tenure_bucket	agent_shift
0	Richard Buchanan	Mason Gupta	Jennifer Nguyen	On Job Training	Morning
1	Vicki Collins	Dylan Kim	Michael Lee	>90	Morning
2	Duane Norman	Jackson Park	William Kim	On Job Training	Evening
3	Patrick Flores	Olivia Wang	John Smith	>90	Evening
4	Christopher Sanchez	Austin Johnson	Michael Lee	0-30	Morning

In [97]:

```
pd.read_sql("""
SELECT *
FROM interactions
LIMIT 5
""", conn)
```

Out[97]:

	unique_id	channel_name	category	sub_category	customer_remarks	issue_rep
0	7e9ae164-6a8b-4521-a2d4-58f7c9fff13f	Outcall	Product Queries	Life Insurance	None	20
1	b07ec1b0-f376-43b6-86df-ec03da3b2e16	Outcall	Product Queries	Product Specific Information	None	20
2	200814dd-27c7-4149-ba2b-bd3af3092880	Inbound	Order Related	Installation/Demo	None	20
3	eb0d3e53-c1ca-42d3-8486-e42c8d622135	Inbound	Returns	Reverse Pickup Enquiry	None	20
4	ba903143-1e54-406c-b969-46c52f92e5df	Inbound	Cancellation	Not Needed	None	20

In [98]:

```
pd.read_sql("""
SELECT unique_id, customer_city
FROM customers
WHERE unique_id = '4c28acf4-2ea4-4be8-b8f1-113e676fc8b7'
""", conn)
```

Out[98]:

	unique_id	customer_city
0	4c28acf4-2ea4-4be8-b8f1-113e676fc8b7	NAGPUR

In [100…

```
customers = pd.read_sql("SELECT * FROM customers", conn)

customers.head()
```

Out[100…

	unique_id	customer_city
0	7e9ae164-6a8b-4521-a2d4-58f7c9fff13f	None
1	b07ec1b0-f376-43b6-86df-ec03da3b2e16	None
2	200814dd-27c7-4149-ba2b-bd3af3092880	None
3	eb0d3e53-c1ca-42d3-8486-e42c8d622135	None
4	ba903143-1e54-406c-b969-46c52f92e5df	None

In [101…

```
customers["customer_city"].value_counts(dropna=False).head(10)
```

Out[101…

```
customer_city
None         68828
HYDERABAD      722
NEW DELHI      688
PUNE           435
MUMBAI         406
BANGALORE      352
CHENNAI        271
KOLKATA        270
LUCKNOW        254
AHMEDABAD      253
Name: count, dtype: int64
```

In [102…

```
orders = pd.read_sql("SELECT * FROM orders", conn)

orders.head()
```

Out[102…

	order_id	order_date_time	item_price	product_category
0	c27c9bb4-fa36-4140-9f1f-21009254ffdb	None	NaN	None
1	d406b0c7-ce17-4654-b9de-f08d421254bd	None	NaN	None
2	c273368d-b961-44cb-beaf-62d6fd6c00d5	None	NaN	None
3	5aed0059-55a4-4ec6-bb54-97942092020a	None	NaN	None
4	e8bed5a9-6933-4aff-9dc6-ccefd7dcde59	None	NaN	None

In [103…

```
orders["product_category"].value_counts(dropna=False).head(10)
```

Out[103…

```
product_category
None                           50480
Electronics                     4706
Lifestyle                       4118
Books & General Merchandise     3323
Mobile                          1758
Home                            1328
Home Appliances                 1300
Furniture                        471
Affiliates                       166
Gift Card                         26
Name: count, dtype: int64
```

In [104…

```
customers.isnull().mean() * 100
```

Out[104…

```
unique_id         0.000000
customer_city    80.119199
dtype: float64
```

In [105…

```
orders.isnull().mean() * 100
```

Out[105…

```
order_id             0.001478
order_date_time     74.564100
item_price          74.575921
product_category    74.590697
dtype: float64
```

In [106…

```
pd.read_sql("SELECT COUNT(*) FROM interactions", conn)
```

Out[106…

	COUNT(*)
0	85907

Observations & Insights

- The missing value analysis reveals that certain columns in the customers and orders tables contain a high percentage of missing values.
- In the customers table, the customer_city column has approximately 80% missing values, indicating that location information is available for only a subset of interactions. This may occur because customer location is not always captured during support interactions or may be stored in external systems.
- In the orders table, the fields order_date_time, item_price, and product_category each contain approximately 74–75% missing values. This suggests that many support interactions are not directly associated with detailed order information, possibly because they relate to general inquiries, technical issues, or other non-transactional support requests.

- These missing values are therefore structural characteristics of the dataset rather than data quality errors. To preserve analytical integrity, missing values will not be artificially filled. Instead, analyses involving order attributes or customer location will be performed only on records where valid values are available.

As a result, the primary analytical focus of this project will be on interaction-level metrics such as issue categories, response times, handling time, and customer satisfaction scores.

Feature Engineering

Feature engineering transforms raw data into variables that improve analytics capabilities.

We will create the following engineered columns:

- Resolution_Time
- Issue_Response_Delay
- Handling_Time_Minutes
- Customer_Satisfaction_Category

These features will help answer important business questions such as:

- How long it takes to resolve issues
- How quickly agents respond
- Whether satisfaction correlates with resolution speed

In [107…

```
# Convert timestamp columns

interactions["issue_reported_at"] = pd.to_datetime(interactions["issue_reported_at"
interactions["issue_responded"] = pd.to_datetime(interactions["issue_responded"])
```

In [108…

```
# Resolution time

interactions["resolution_time"] = (
    interactions["issue_responded"] - interactions["issue_reported_at"]
).dt.total_seconds() / 60
```

In [109…

```
# Handling time in minutes

interactions["handling_time_minutes"] = interactions["connected_handling_time"] / 6
```

In [110…

```
# Customer satisfaction category

interactions["customer_satisfaction_category"] = pd.cut(
    interactions["csat_score"],
    bins=[0,2,3,5],
    labels=["Low","Medium","High"]
)
```

In [112…

```
interactions.head()
```

Out[112…

	unique_id	channel_name	category	sub_category	customer_remarks	issue_rep
0	7e9ae164-6a8b-4521-a2d4-58f7c9fff13f	Outcall	Product Queries	Life Insurance	None	20
1	b07ec1b0-f376-43b6-86df-ec03da3b2e16	Outcall	Product Queries	Product Specific Information	None	20
2	200814dd-27c7-4149-ba2b-bd3af3092880	Inbound	Order Related	Installation/Demo	None	20
3	eb0d3e53-c1ca-42d3-8486-e42c8d622135	Inbound	Returns	Reverse Pickup Enquiry	None	20
4	ba903143-1e54-406c-b969-46c52f92e5df	Inbound	Cancellation	Not Needed	None	20

In [113…

```
interactions.to_sql(
    "interactions",
    conn,
    if_exists="replace",
    index=False
)
```

Out[113…

```
85907
```

In [114…

```
pd.read_sql("""
SELECT *
FROM interactions
LIMIT 5
""", conn)
```

Out[114…

	unique_id	channel_name	category	sub_category	customer_remarks	issue_rep
0	7e9ae164-6a8b-4521-a2d4-58f7c9fff13f	Outcall	Product Queries	Life Insurance	None	20
1	b07ec1b0-f376-43b6-86df-ec03da3b2e16	Outcall	Product Queries	Product Specific Information	None	20
2	200814dd-27c7-4149-ba2b-bd3af3092880	Inbound	Order Related	Installation/Demo	None	20
3	eb0d3e53-c1ca-42d3-8486-e42c8d622135	Inbound	Returns	Reverse Pickup Enquiry	None	20
4	ba903143-1e54-406c-b969-46c52f92e5df	Inbound	Cancellation	Not Needed	None	20

In [122…

```
interactions.to_sql(
    "interactions",
    conn,
    if_exists="replace",
    index=False
)
```

Out[122…

```
85907
```

In [124…

```
pd.read_sql("PRAGMA table_info(interactions)", conn)
```

Out[124…

	cid	name	type	notnull	dflt_value	pk
0	0	unique_id	TEXT	0	None	0
1	1	channel_name	TEXT	0	None	0
2	2	category	TEXT	0	None	0
3	3	sub_category	TEXT	0	None	0
4	4	customer_remarks	TEXT	0	None	0
5	5	issue_reported_at	TIMESTAMP	0	None	0
6	6	issue_responded	TIMESTAMP	0	None	0
7	7	connected_handling_time	REAL	0	None	0
8	8	csat_score	INTEGER	0	None	0
9	9	has_customer_remark	INTEGER	0	None	0
10	10	resolution_time	REAL	0	None	0
11	11	handling_time_minutes	REAL	0	None	0
12	12	customer_satisfaction_category	TEXT	0	None	0

Creating Analytical Summary Table

In real analytics pipelines, raw operational tables are rarely used directly for reporting.

Instead, data teams create analytical summary tables that:

- contain cleaned features
- include engineered variables
- remove unnecessary columns
- optimize queries for BI tools

The summary table created here will be called: `customer_support_summary`

This table will serve as the primary dataset for statistical analysis and dashboard development.

In [126…

```
summary = pd.read_sql("""

SELECT
    -- Interaction identifier
    i.unique_id,

    -- Interaction details
    i.category,
    i.sub_category,
    i.channel_name,
```

```
    -- Product information
    o.product_category,

    -- Engineered metrics
    i.resolution_time,
    i.handling_time_minutes,
    i.csat_score,
    i.customer_satisfaction_category,
    i.has_customer_remark

FROM interactions i

LEFT JOIN orders o
ON i.unique_id = o.order_id

""", conn)

summary.head()
```

Out[126…

	unique_id	category	sub_category	channel_name	product_category	resolutio
0	7e9ae164-6a8b-4521-a2d4-58f7c9fff13f	Product Queries	Life Insurance	Outcall	None	
1	b07ec1b0-f376-43b6-86df-ec03da3b2e16	Product Queries	Product Specific Information	Outcall	None	
2	200814dd-27c7-4149-ba2b-bd3af3092880	Order Related	Installation/Demo	Inbound	None	
3	eb0d3e53-c1ca-42d3-8486-e42c8d622135	Returns	Reverse Pickup Enquiry	Inbound	None	
4	ba903143-1e54-406c-b969-46c52f92e5df	Cancellation	Not Needed	Inbound	None	

In [127…

```
summary.to_sql(
    "customer_support_summary",
    conn,
    if_exists="replace",
    index=False
)
```

Out[127… 85907

Confirm Summary Table Creation

We verify that the summary table was successfully written to the database.

In [128…
```
pd.read_sql(
    "SELECT COUNT(*) FROM customer_support_summary",
    conn
)
```

Out[128…

	COUNT(*)
0	85907

In [129…
```
pd.read_sql("""
PRAGMA table_info(customer_support_summary);
""", conn)
```

Out[129…

	cid	name	type	notnull	dflt_value	pk
0	0	unique_id	TEXT	0	None	0
1	1	category	TEXT	0	None	0
2	2	sub_category	TEXT	0	None	0
3	3	channel_name	TEXT	0	None	0
4	4	product_category	TEXT	0	None	0
5	5	resolution_time	REAL	0	None	0
6	6	handling_time_minutes	REAL	0	None	0
7	7	csat_score	INTEGER	0	None	0
8	8	customer_satisfaction_category	TEXT	0	None	0
9	9	has_customer_remark	INTEGER	0	None	0

In [130…
```
df = pd.read_sql("SELECT * FROM customer_support_summary", conn)
df.to_csv("Data/customer_support_summary.csv", index=False)
```

In [131…
```
df = pd.read_csv("data/customer_support_summary.csv")
```

In [132…
```
df.head()
```

Out[132…

	unique_id	category	sub_category	channel_name	product_category	resolutio
0	7e9ae164-6a8b-4521-a2d4-58f7c9fff13f	Product Queries	Life Insurance	Outcall	NaN	
1	b07ec1b0-f376-43b6-86df-ec03da3b2e16	Product Queries	Product Specific Information	Outcall	NaN	
2	200814dd-27c7-4149-ba2b-bd3af3092880	Order Related	Installation/Demo	Inbound	NaN	
3	eb0d3e53-c1ca-42d3-8486-e42c8d622135	Returns	Reverse Pickup Enquiry	Inbound	NaN	
4	ba903143-1e54-406c-b969-46c52f92e5df	Cancellation	Not Needed	Inbound	NaN	

Observations & Insights

Review the sample records from the summary table.

Confirm that

- engineered features are correctly calculated
- unnecessary columns have been removed
- the dataset is ready for statistical analysis

Outcome

This notebook successfully completed EDA Phase 1 and foundational data engineering tasks.

Key achievements include:

- SQL-based database inspection
- schema validation
- missing value analysis
- duplicate detection
- categorical data exploration

- feature engineering
- analytical summary table creation

The customer_support_summary table created in this notebook will now serve as the central dataset for statistical analysis and visualization in the next stage of the project.

In [ ]:

```

```